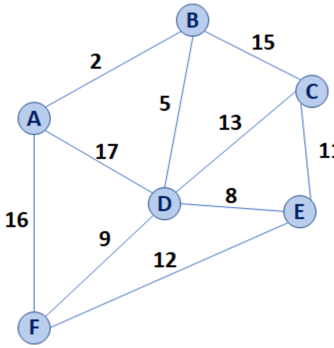
CS 225

#38: Dijkstra's Algorithm

May 20-22, 2026 V. Kindratenko

Prim's Algorithm (Minimum Spanning Tree)



Pseudocode for Prim's MST Algorithm

```
PrimMST(G, s):
  Input: G, Graph;
         s, vertex in G, starting vertex of algorithm
  Output: T, a minimum spanning tree (MST) of G

  foreach (Vertex v : G):
    d[v] = +inf
    p[v] = NULL
  d[s] = 0

  PriorityQueue Q   // min distance, defined by d[v]
  Q.buildHeap(G.vertices())
  Graph T           // "labeled set"

  repeat n times:
    Vertex m = Q.removeMin()
    T.add(m)
    foreach (Vertex v : neighbors of m not in T):
      if cost(v, m) < d[v]:
        d[v] = cost(v, m)
        p[v] = m

  return T
```

	Adj. Matrix	**Adj. List**
Heap		
Unsorted Array		

Running Time of MST Algorithms

- Kruskal's Algorithm:
- Prim's Algorithm:

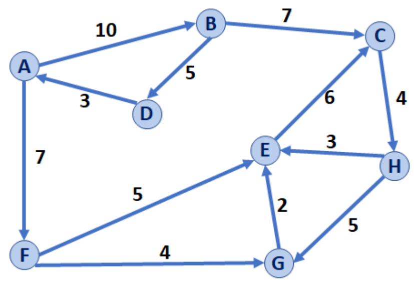
Q: What must be true about the connectivity of a graph when running an MST algorithm?

...what does this imply about the relationship between **n** and **m**?

Q: Suppose we built a new heap that optimized the decrease-key operation, where decreasing the value of a key in a heap updates the heap in amortized constant time, or O(1)*. How does that change Prim's Algorithm runtime?

Shortest Path Home:

Dijkstra's Algorithm (Single Source Shortest Path)



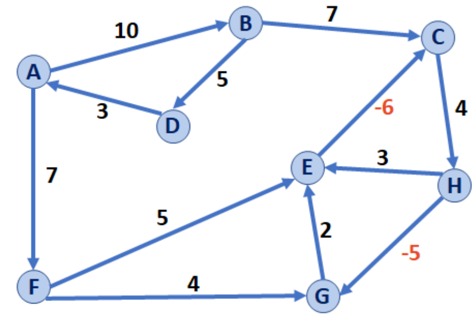
Dijkstra's Algorithm Overview:

- The overall logic is the same as Prim's Algorithm
- We will modify the code in only two places – both involving the update to the distance metric.
- The result is a directed acyclic graph or DAG

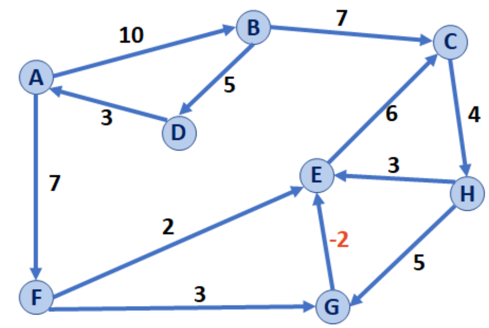
Pseudocode for Dijkstra's SSSP Algorithm

```
DijkstraSSSP(G, s):
  Input: G, Graph;
         s, vertex in G, starting vertex of algorithm
  Output: T, DAG with shortest paths (and distances) to s

  foreach (Vertex v : G):
    d[v] = +inf
    p[v] = NULL
  d[s] = 0

  PriorityQueue Q   // min distance, defined by d[v]
  Q.buildHeap(G.vertices())
  Graph T           // "labeled set"

  repeat n times:
    Vertex m = Q.removeMin()
    T.add(m)
    foreach (Vertex v : neighbors of m not in T):
      if ________________________ < d[v]:
        d[v] = _________________________
        p[v] = m

  return T
```

Dijkstra: What if we have a negative-weight cycle?



Dijkstra: What if we have a minimum-weight edge, without having a negative-weight cycle?



Dijkstra makes an assumption:

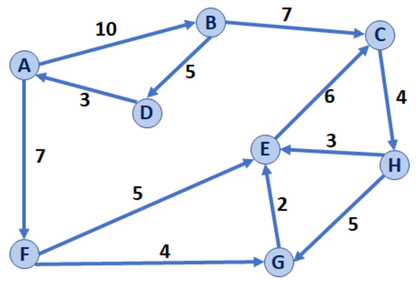
Dijkstra: What is the running time?

CS 225

#39: Dijkstra's + Shortest Path

May 20-22, 2026 V. Kindratenko

Dijkstra's Algorithm (Single Source Shortest Path)



Dijkstra's Algorithm Overview:

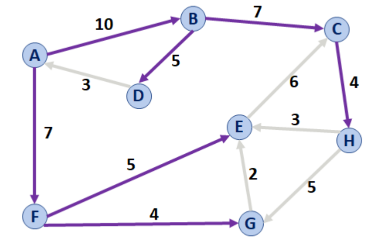
- The overall logic is the same as Prim's Algorithm
- We will modify the code in only two places – both involving the update to the distance metric.
- The result is a directed acyclic graph or DAG

Pseudocode for Dijkstra's SSSP Algorithm

```
DijkstraSSSP(G, s):
  Input: G, Graph;
         s, vertex in G, starting vertex of algorithm
  Output: T, DAG with shortest paths (and distances) to s

  foreach (Vertex v : G):
    d[v] = +inf
    p[v] = NULL
  d[s] = 0

  PriorityQueue Q   // min distance, defined by d[v]
  Q.buildHeap(G.vertices())
  Graph T           // "labeled set"

  repeat n times:
    Vertex m = Q.removeMin()
    T.add(m)
    foreach (Vertex v : neighbors of m not in T):
      if cost(u, v) + d[u] < d[v]:
        d[v] = cost(u, v) + d[u]
        p[v] = m

  return T
```

Backtracking in Dijkstra

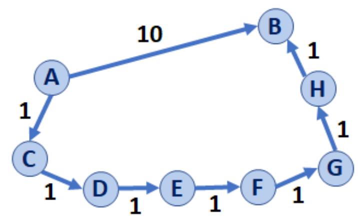
Dijkstra's Algorithm gives us the shortest path from a single source to every connected vertex:



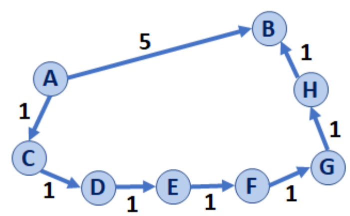
	A	B	C	D	E	F	G	H
p	NULL	A	B	B	F	A	F	C
d	0	10	17	15	12	7	11	21

Examples: How is a single heavy-weight path vs. many light-weight paths handled?

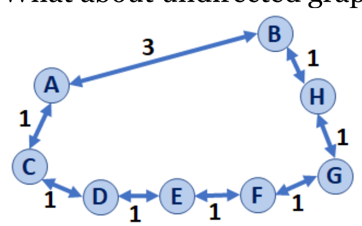
Ex 1:



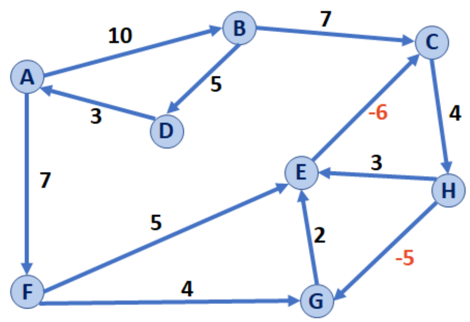
Ex 2:



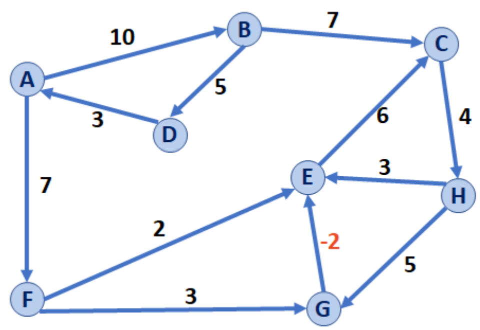
What about undirected graphs?



Dijkstra: What if we have a negative-weight cycle?



Dijkstra: What if we have a minimum-weight edge, without having a negative-weight cycle?



Dijkstra makes an assumption:

Dijkstra: What is the running time?

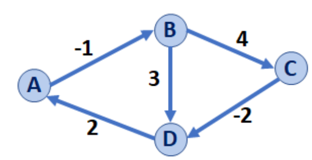
Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alterative to Dijkstra in the presence of negative-weight edges (but <u>not</u> negative weight cycles).

Pseudocode for Floyd-Warshall's Algorithm

```
FloydWarshall(G):
  Input: G, Graph;
  Output: d, an adjacency matrix of distances between all
          vertex pairs

  Let d be a adj. matrix initialized to +inf
  foreach (Vertex v : G):
    d[v][v] = 0
  foreach (Edge (u, v) : G):
    d[u][v] = cost(u, v)

  foreach (Vertex w : G):
    foreach (Vertex u : G):
      foreach (Vertex v : G):
        if d[u, v] > d[u, w] + d[w, v]:
          d[u, v] = d[u, w] + d[w, v]

  return d
```

Running Floyd-Warshall's Algorithm



#40: Floyd-Warshall's Algorithm

May 20-22, 2026 V. Kindratenko

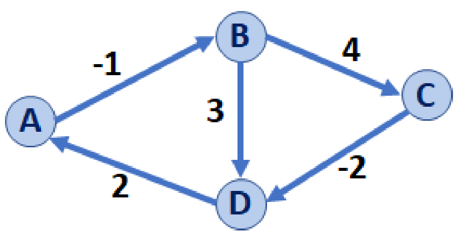
Floyd-Warshall Algorithm

Floyd-Warshall's Algorithm is an alternative to Dijkstra in the presence of negative-weight edges (but <u>not</u> negative weight cycles).

Pseudocode for Floyd-Warshall's Algorithm

```
FloydWarshall(G):
  Input: G, Graph;
  Output: d, an adjacency matrix of distances between all
          vertex pairs

  Let d be a adj. matrix initialized to +inf
  foreach (Vertex v : G):
    d[v][v] = 0
  foreach (Edge (u, v) : G):
    d[u][v] = cost(u, v)

  foreach (Vertex w : G):
    foreach (Vertex u : G):
      foreach (Vertex v : G):
        if d[u, v] > d[u, w] + d[w, v]:
          d[u, v] = d[u, w] + d[w, v]

  return d
```

Running Floyd-Warshall:



	A	B	C	D
A				
B				
C				
D				

Comparison of Graph Algorithms:

Implementations
- Edge List
- Adjacency Matrix
- Adjacency List

Traversals
- Breadth First
- Depth First

Minimum Spanning Tree
- Kruskal's Algorithm
- Prim's Algorithm

Shortest Path
- Dijkstra's Algorithm
- Floyd-Warshall's Algorithm

CS 225 – Things To Be Doing:

1. mp7 due on Monday May 25
2. lab_ml due on Sunday May 24
3. Quiz 7 this Thursday

Good luck on all your finals! :)